

Laboratório de Concorrência · Aula 10

Gabriel Galhardone Felipe · RA 240212922 · Turma EsoftB

Gerado em 09/10/2026, 20:11:16 · tempo registrado no cronômetro: 12:25

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

A sincronização de cooperação acontece quando pessoas trabalham juntas para alcançar um objetivo em comum. Por exemplo, várias pessoas se ajudam para organizar uma festa.

Já a sincronização de competição acontece quando pessoas ou grupos disputam para alcançar um objetivo, tentando obter um resultado melhor que os outros. Por exemplo, dois corredores disputam uma corrida para ver quem chega primeiro.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

Espero que o programa imprima 2.000.000, pois são duas threads somando 1 ao contador um milhão de vezes cada. Porém, devido à condição de corrida, o resultado pode ser menor que o esperado.

1ª execução (n = 1.000.000)

1.654.328

2ª execução (n = 1.000.000)

1.782.456

3ª execução (n = 1.000.000)

1.598.721

Saída do programa

esperado: 2000000 obtido: 1782456

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não, os valores obtidos podem ser menores que o esperado e não foram necessariamente iguais nas três execuções. Eles mudam porque as duas threads acessam o mesmo contador ao mesmo tempo, causando uma condição de corrida e fazendo com que algumas somas sejam perdidas.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Ler: as duas threads leem o contador com valor 5 ao mesmo tempo.

Somar: cada thread soma 1 ao valor lido, chegando a 6.

Gravar: a primeira thread grava 6 e, em seguida, a segunda também grava 6.

Assim, o contador fica com valor 6 em vez de 7. Uma das atualizações se perdeu porque as duas threads usaram o mesmo valor inicial.

1.3 Com $n = 1000$, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com $n = 1000$, o resultado pode ter ficado mais próximo do esperado, que é 2000, ou até ter sido exatamente 2000. Esse erro é perigoso porque a condição de corrida acontece de forma imprevisível: nos testes, o programa pode funcionar corretamente, mas em produção, com mais usuários e threads executando ao mesmo tempo, algumas atualizações podem se perder e causar resultados incorretos.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

ABABAB $\rightarrow$ 6

Saída do programa

AAABBB $\rightarrow$ 8 BBBAAB $\rightarrow$ 7 ABABAB $\rightarrow$ 6 Ordens possíveis: 20 TOTAL = 4: 9 ordens, por exemplo AABBBB TOTAL = 6: 9 ordens, por exemplo AABABB TOTAL = 7: 1 ordens, por exemplo BBBAAB TOTAL = 8: 1 ordens, por exemplo AAABBB

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

São possíveis 20 ordens, que geram 4 resultados diferentes: 4, 6, 7 e 8.

Apenas 1 ordem produz o resultado correto (8): AAABBB.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Escolhendo a ordem AABBBB:

A lê: TOTAL = 3.

A calcula: $3 + 1 = 4$.

B lê: TOTAL = 3.

B calcula: $3 \times 2 = 6$.

B grava: TOTAL = 6.

A grava: TOTAL = 4.

B grava novamente: TOTAL = 4.

O resultado final é 4, porque a operação de B foi sobrescrita pela gravação de A. A atualização de B se perdeu devido à condição de corrida.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Acredito que a operação atômica (AtomicInteger) será a mais rápida, pois realiza a atualização de forma segura. Nenhuma das três versões deve apresentar resultados incorretos, pois todas garantem a sincronização e evitam a condição de corrida.

Saída do programa

```
esperado: 2000000 sem sincronizacao: 1245837 em 42 ms synchronized: 2000000 em 286 ms  
AtomicInteger: 2000000 em 105 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

As versões synchronized e AtomicInteger deram o valor correto de 2.000.000. A versão sem sincronização pode apresentar um valor menor devido à condição de corrida.

As versões corretas costumam ser mais lentas porque precisam controlar o acesso ao contador compartilhado, evitando que as duas threads alterem o valor ao mesmo tempo. Esse controle gera um custo extra de processamento.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não. Um AtomicInteger em cada saldo garante que cada operação individual seja segura, mas não garante que a transferência inteira aconteça de forma atômica. Seria necessário usar sincronização para bloquear as duas contas durante a transferência, garantindo que o dinheiro seja retirado de uma conta e depositado na outra sem inconsistências.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Não. As mensagens “A terminou” e “B terminou” provavelmente não vão aparecer, porque ocorre um impasse (deadlock): a thread A segura o recurso X e espera Y, enquanto a thread B segura Y e espera X. Assim, nenhuma consegue continuar.

Saídas (antes e depois da correção)

```
IMPASSE: 2 threads presas  
A terminou B terminou sem impasse
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

A thread A ficou com o recurso X e esperava pelo recurso Y. Ao mesmo tempo, a thread B ficou com o recurso Y e esperava pelo recurso X. Como cada uma estava segurando um recurso que a outra precisava, nenhuma conseguia continuar, causando o impasse (deadlock).

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Pegar os recursos sempre na mesma ordem elimina o impasse porque impede que uma thread segure um recurso enquanto espera por outro em uma ordem inversa. Assim, evita-se que uma fique esperando pela outra indefinidamente.

Exemplo real: em um sistema bancário, duas transferências entre contas podem tentar acessar os mesmos saldos ao mesmo tempo. Se ambas bloquearem as contas sempre na mesma ordem, evita-se que uma transferência fique esperando pela outra em um impasse.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Com 3 trabalhadores: espero cerca de 200 ms, pois cada trabalhador executa 2 tarefas de 100 ms.

Com 1 trabalhador: espero cerca de 600 ms, pois ele executa as 6 tarefas em sequência.

Com 6 trabalhadores: espero cerca de 100 ms, pois cada trabalhador executa uma tarefa ao mesmo tempo.

Tempo com 1 trabalhador

600 ms

Tempo com 3 trabalhadores

200 ms

Tempo com 6 trabalhadores

100 ms

Saída do programa

```
1 trabalhadores, 6 tarefas de 100 ms: levou 600ms 3 trabalhadores, 6 tarefas de 100 ms: levou 200ms 6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```

5.1 Explique os três tempos. Por que este programa não precisa de `synchronized` nem de trava?

Com 1 trabalhador, o tempo é de aproximadamente 600 ms, pois ele executa as seis tarefas em sequência. Com 3 trabalhadores, leva cerca de 200 ms, porque cada um realiza duas tarefas. Com 6 trabalhadores, leva aproximadamente 100 ms, pois todos executam uma tarefa ao mesmo tempo.

O programa não precisa de `synchronized` nem de travas porque as tarefas são distribuídas por um canal (channel) do Go, que permite a comunicação entre as goroutines sem compartilhar diretamente uma variável que todas modificam.

Saídas (antes e depois da troca)

```
Antes da troca — make(chan int)
```

```
Escrita
```

```
fatal error: all goroutines are asleep - deadlock!
```

```
goroutine 1 [chan send]: main.main() /tmp/sandbox.../main.go:9 +0x...
```

```
Depois da troca — make(chan int, 1)
```

```
Escrita
```

```
1
```

Explicação: o canal sem buffer bloqueia o envio até que outra goroutine receba a mensagem, causando um deadlock. Com buffer de tamanho 1, o canal armazena o valor 1, permitindo que o envio termine e que o programa imprima 1.

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O primeiro programa trava porque o canal não tem espaço para armazenar mensagens. Quando a função tenta enviar o valor 1, fica esperando outra goroutine receber a mensagem, mas ninguém está disponível para recebê-la.

Ao usar `make(chan int, 1)`, o canal passa a ter espaço para armazenar uma mensagem. Assim, o valor 1 fica guardado até ser recebido, e o programa consegue continuar e imprimir o resultado.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Acredito que o programa não vai compilar, porque o Rust não permite que uma thread altere diretamente uma variável de main sem garantir o compartilhamento seguro entre as threads. O compilador deve apresentar um erro indicando que a variável não pode ser emprestada dessa forma com segurança.

Saídas (o erro e o resultado com move)

```
error[E0373]: closure may outlive the current function, but it borrows contador, which is
owned by the current function help: to force the closure to take ownership of contador (and
any other referenced variables), use the move keyword
```

```
Dps
1
```

Saída do programa

```
contador = 400000
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Isso acontece porque o `move` transfere a propriedade da variável `contador` para a thread. A thread altera sua própria versão do número, enquanto a variável original de main continua com o valor 0. Por isso, o programa imprime 0. Para compartilhar e alterar o mesmo contador entre várias threads, é necessário usar ferramentas como `Arc` e `Mutex`.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, o erro foi descoberto durante a execução do programa, quando o contador apresentou valores incorretos por causa da condição de corrida. Em Rust, o compilador identificou o problema antes da execução.

Vantagem: Rust ajuda a evitar erros de concorrência antes de o programa rodar.

Custo: quem programa precisa aprender e usar mecanismos como Arc e Mutex, o que pode tornar o código mais complexo.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

Acredito que a ordem será A, D, C, B, dependendo de como o código foi escrito. Em JavaScript, a ordem de execução depende de quais instruções são síncronas, quais usam setTimeout e quais usam promessas. Se você enviar o código, posso indicar a ordem exata.

Saída do programa

```
A D C B E: timer de 10 ms rodou depois de 300 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

As letras saíram na ordem A, D, C, B. Depois, apareceu a mensagem do timer E.

A letra B, mesmo com tempo 0, saiu depois de D e C porque o setTimeout não executa imediatamente: ele agenda a função para depois que o código atual terminar. Já C é uma promessa (Promise), que é executada antes dos timers. Por isso, D e C aparecem antes de B.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O timer de 10 ms demorou mais porque a função ocupado(300) bloqueou a única thread por cerca de 300 ms, impedindo que o laço de eventos executasse outras tarefas.

Em um servidor Node.js, isso significa que um cálculo pesado pode atrasar as respostas de muitos usuários ao mesmo tempo, deixando o servidor lento. Para evitar isso, é possível usar threads de trabalho (Worker Threads) ou dividir o cálculo em tarefas menores, permitindo que o servidor continue atendendo às requisições.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode acontecer uma condição de corrida: as duas pessoas tentam comprar o último ingresso ao mesmo tempo e o sistema pode vender o mesmo lugar para ambas.

Para evitar isso, usaria uma transação no banco de dados com bloqueio ou atualização atômica, garantindo que apenas uma compra seja confirmada. Assim, a primeira compra reserva o ingresso e a segunda recebe uma mensagem informando que ele esgotou.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi perceber que pequenos erros de sincronização podem causar grandes problemas, mesmo quando o código parece estar correto.